

BRIEFINGS

black hat

Breaking Hardware CFI with Sigreturn

Omri Ben-Bassat , Prof. Noam Rinetzky, Prof. Sharon Shoham, Prof. Adam Morrison

Tel Aviv University

\$ whoami



Omri Ben-Bassat
*Vulnerability Researcher
& MSc Candidate*



Prof. Noam Rinetzky



Prof. Sharon Shoham
Buchbinder



Prof. Adam Morrison

SROP/BTI

- A novel ARM64 hardware-CFI bypass
- Breaks BTI even with PAC enabled
- Demonstrated on Ubuntu 26.04 and Android 17

Control Flow Integrity (CFI)

- Modern class of mitigations
- Restricts a program's execution
- Only intended control-flow paths

ARM64 *Pointer Authentication Code (PAC)*

- ARM mechanism for authenticating pointers
- Can protect both forward and backward edges
- Linux deploys PAC as backward-edge CFI
- Signs return addresses (`pac-ret`)
- Aims to mitigate ROP

PACIxSP + AUTIxSP

```
pwndbg> disas main
Dump of assembler code for function main:
0x0000000000000828 <+0>:    paciasp
0x000000000000082c <+4>:    stp     x29, x30, [sp, #-32]!
0x0000000000000830 <+8>:    mov     x29, sp
0x0000000000000834 <+12>:   str     w0, [sp, #28]
0x0000000000000838 <+16>:   str     x1, [sp, #16]
0x000000000000083c <+20>:   adrp    x0, 0x0
0x0000000000000840 <+24>:   add     x0, x0, #0x878
0x0000000000000844 <+28>:   bl      0x6c0 <printf@plt>
0x0000000000000848 <+32>:   mov     w0, #0x0                                // #0
0x000000000000084c <+36>:   ldp     x29, x30, [sp], #32
0x0000000000000850 <+40>:   autiasp
0x0000000000000854 <+44>:   ret
End of assembler dump.
```

gcc ... -mbranch-protection=pac-ret

PACIxSP + AUTIxSP

```
pwndbg> disas main
Dump of assembler code for function main:
0x0000000000000828 <+0>:    paciasp  ←
0x000000000000082c <+4>:    stp     x29, x30, [sp, #-32]!
0x0000000000000830 <+8>:    mov     x29, sp
0x0000000000000834 <+12>:   str     w0, [sp, #28]
0x0000000000000838 <+16>:   str     x1, [sp, #16]
0x000000000000083c <+20>:   adrp    x0, 0x0
0x0000000000000840 <+24>:   add     x0, x0, #0x878
0x0000000000000844 <+28>:   bl      0x6c0 <printf@plt>
0x0000000000000848 <+32>:   mov     w0, #0x0                                // #0
0x000000000000084c <+36>:   ldp     x29, x30, [sp], #32
0x0000000000000850 <+40>:   autiasp  ←
0x0000000000000854 <+44>:   ret
End of assembler dump.
```

gcc ... -mbranch-protection=pac-ret

ARM64 *Branch Target Identification (BTI)*

- Aims to mitigate JOP
- Forward-edge CFI
- Compiler plants *Landing Pads (LP)*
- LPs are the only valid branch targets
- Enforced in HW

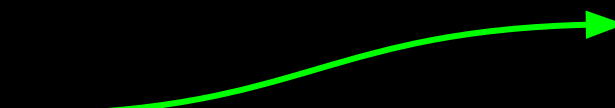
Threat Model

- Target is Linux user mode program
- Latest OS on ARMv8.5-A+
- `-mbranch-protection=standard` (`bti+pac-ret`)
=> No ROP / JOP
- Memory corruption
- ASLR derandomization

BTI implementation

`blr xN`

`pstate.btype := 0b11`



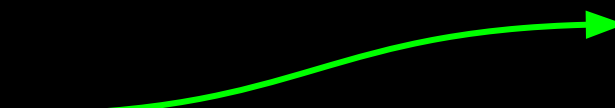
```
// func A
bti // Landing Pad
...
// JOP gadget1
add x0, x1, x2
br xN
...
ret
```

BTI implementation

`blr xN`

`pstate.btype := 0b11`

- `bti`
 - `pstate.btype := 0b00`



```
// func A
bti // Landing Pad
...
// JOP gadget1
add x0, x1, x2
br xN
...
ret
```

BTI implementation

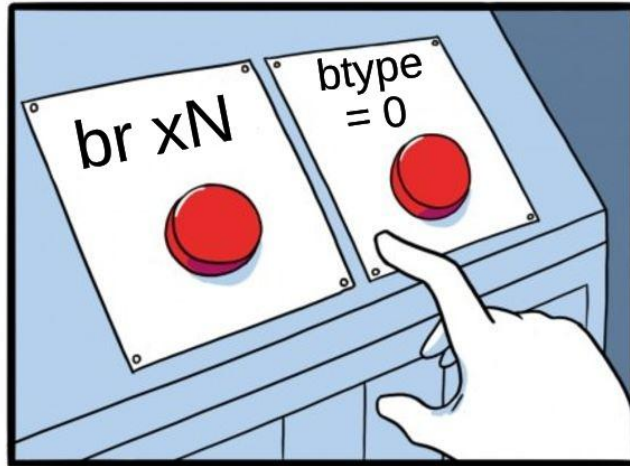
`blr xN`

`pstate.btype := 0b11`

- `bti`
 - `pstate.btype := 0b00`
- `add x0, x1, x2`
 - `pstate.btype != 0 => Branch Target Exception🔥`

```
// func A
bti // Landing Pad
...
// JOP gadget1
add x0, x1, x2
br xN
...
ret
```

<https://developer.arm.com/documentation/102433/0200/Jump-oriented-programming>



No JOP? Just Ask the Kernel!

- `rt_sigreturn`
- POSIX signal-resumption mechanism
 - `x0–x30`, `SP`, `PC`, ...
 - `PSTATE`
- Resumes execution at the restored `PC`
- No BTI landing-pad validation

Sigreturn: A BTI Bypass by Design



user

kernel

```
// func A
...
// JOP gadget1
add x0, x1, x2
br xN
```



Sigreturn: A BTI Bypass by Design



user

kernel

Crafted

rt_sigframe

...
regs[31]
sp
pc = &gadget1
pstate.btype=0

```
// func A
...
// JOP gadget1
add x0, x1, x2
br xN
```


Sigreturn: A BTI Bypass by Design



user

kernel

Crafted
rt_sigframe

...
regs[31]
sp
pc = &gadget1
pstate.btype=0

```
// func A
...
// JOP gadget1
add x0, x1, x2
br xN
```

Sigreturn: A BTI Bypass by Design



user

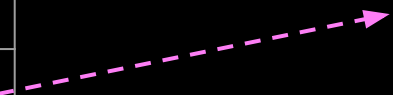
kernel

Crafted
rt_sigframe



`__kernel_rt_sigreturn`

...
regs[31]
sp
pc = &gadget1
pstate.btype=0



```
// func A
...
// JOP gadget1
add x0, x1, x2
br xN
```

Sigreturn: A BTI Bypass by Design



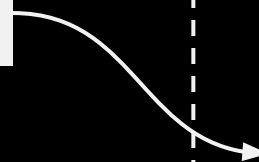
user

kernel

Crafted
rt_sigframe

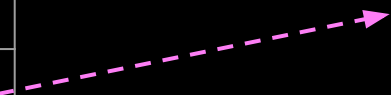


`__kernel_rt_sigreturn`



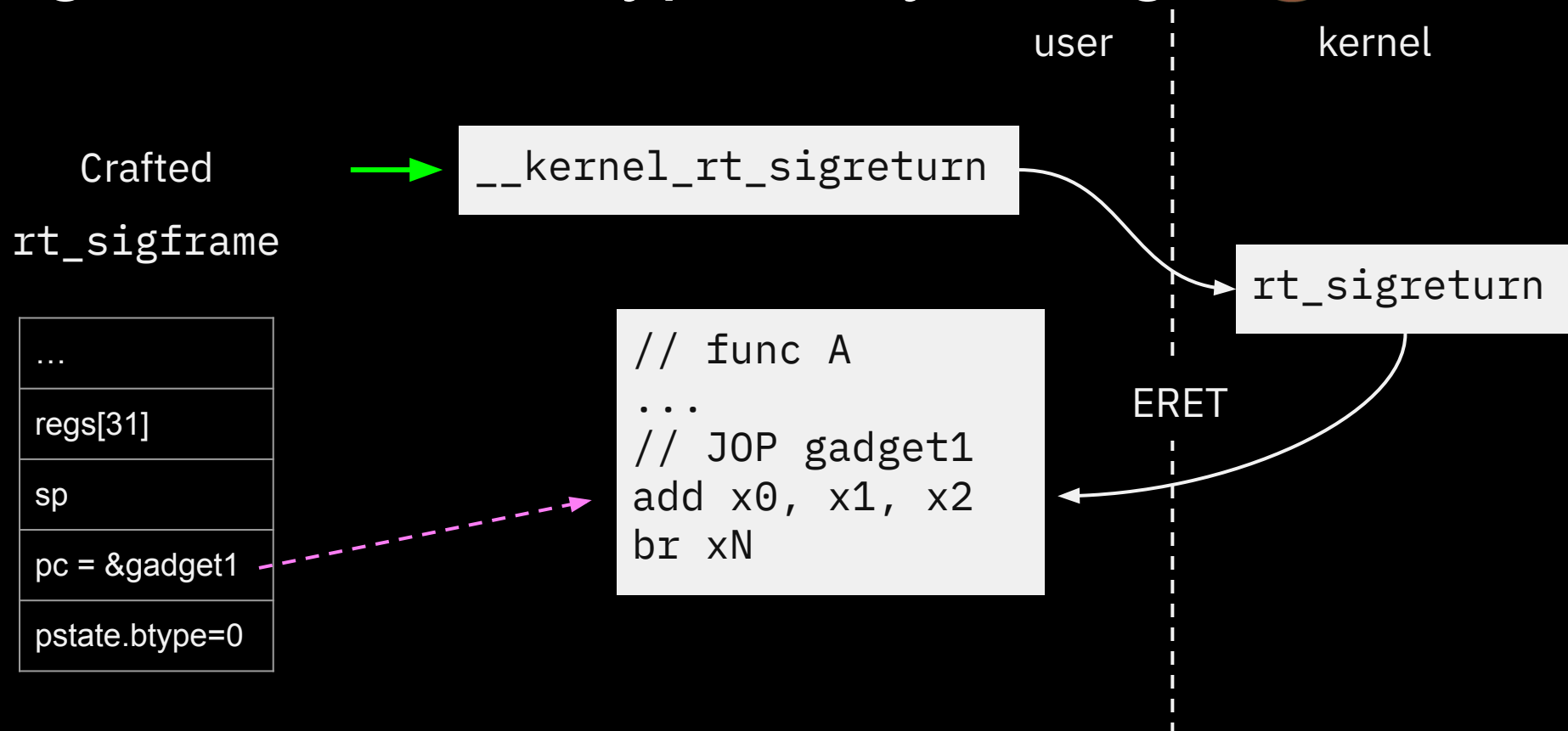
`rt_sigreturn`

...
regs[31]
sp
pc = &gadget1
pstate.btype=0

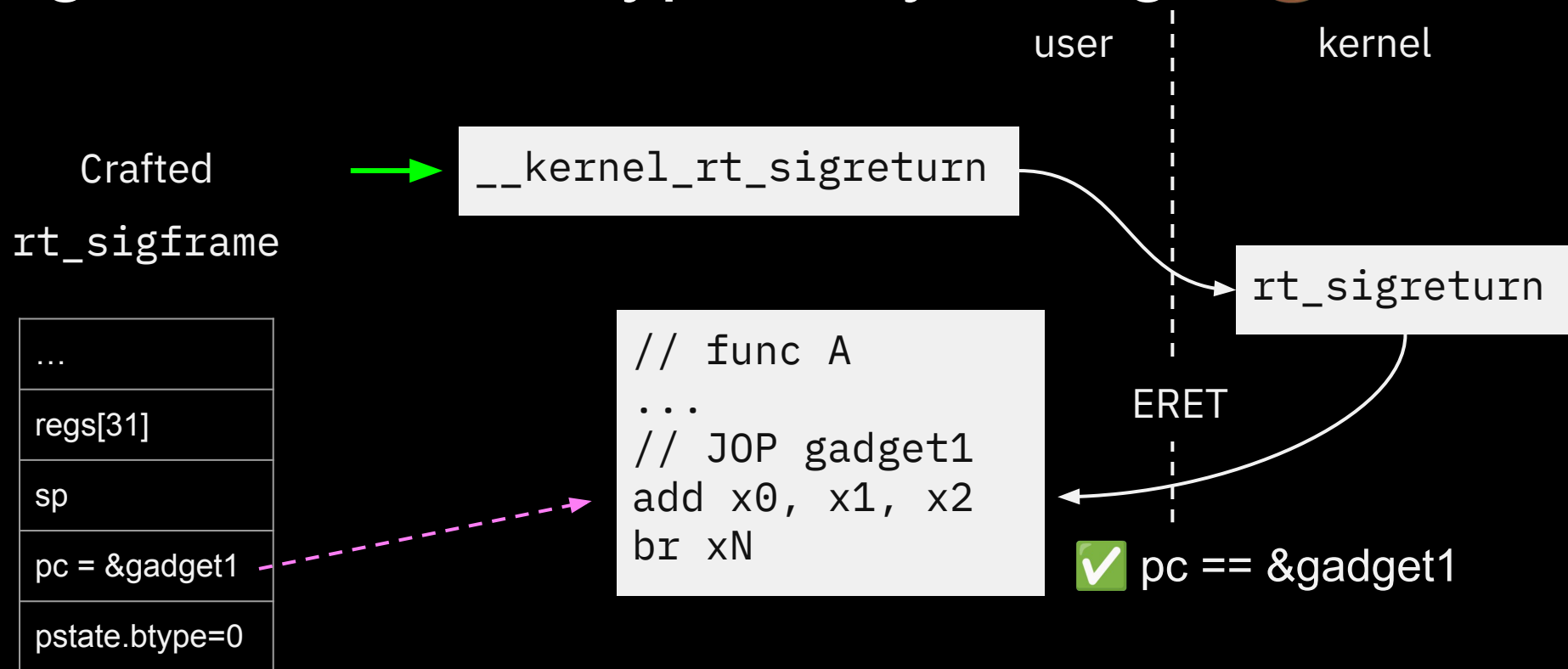


```
// func A
...
// JOP gadget1
add x0, x1, x2
br xN
```

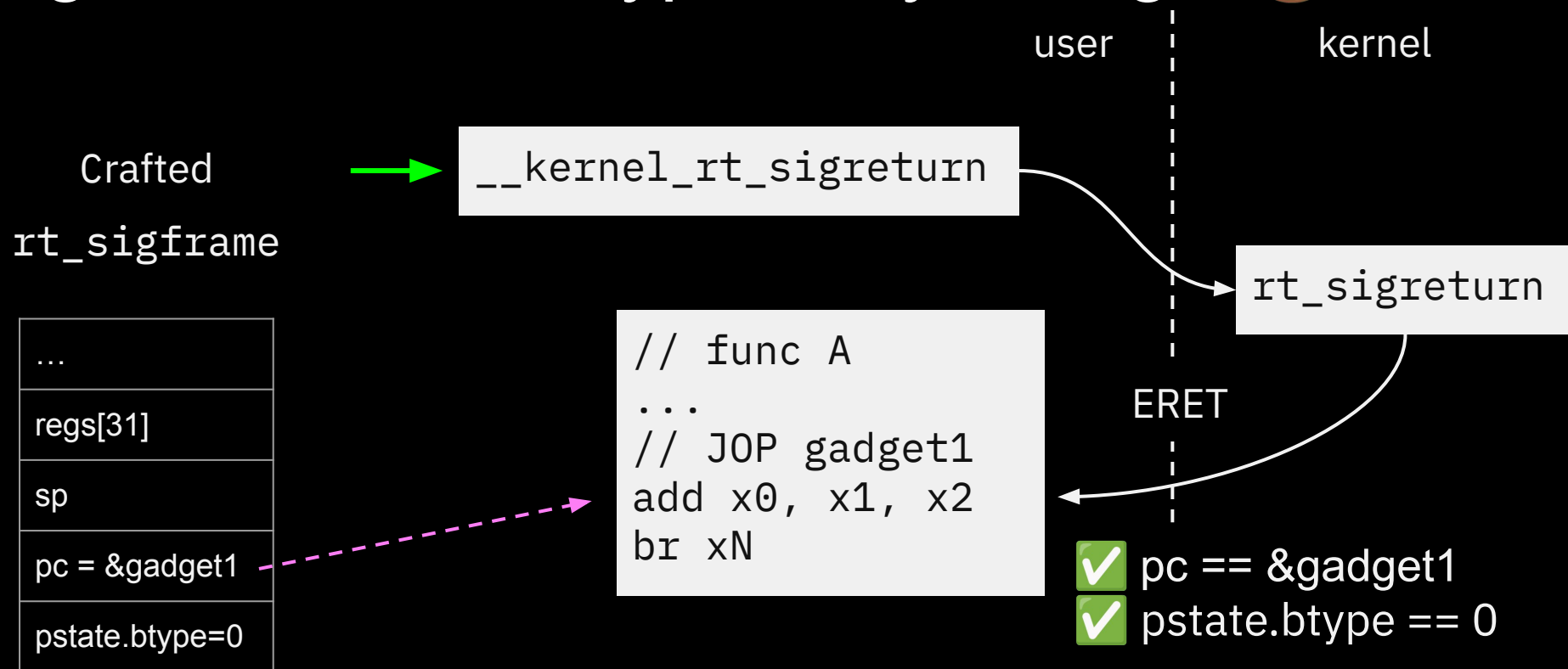
Sigreturn: A BTI Bypass by Design



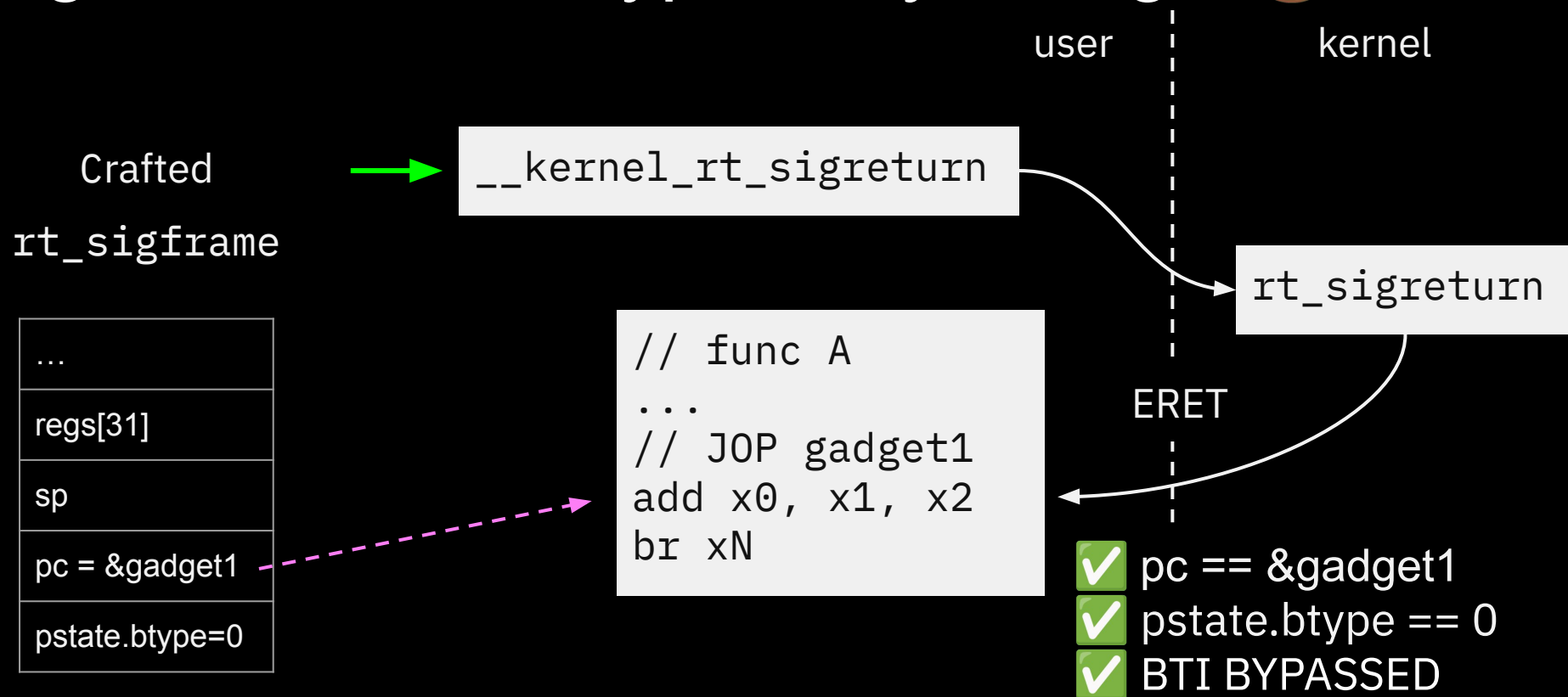
Sigreturn: A BTI Bypass by Design



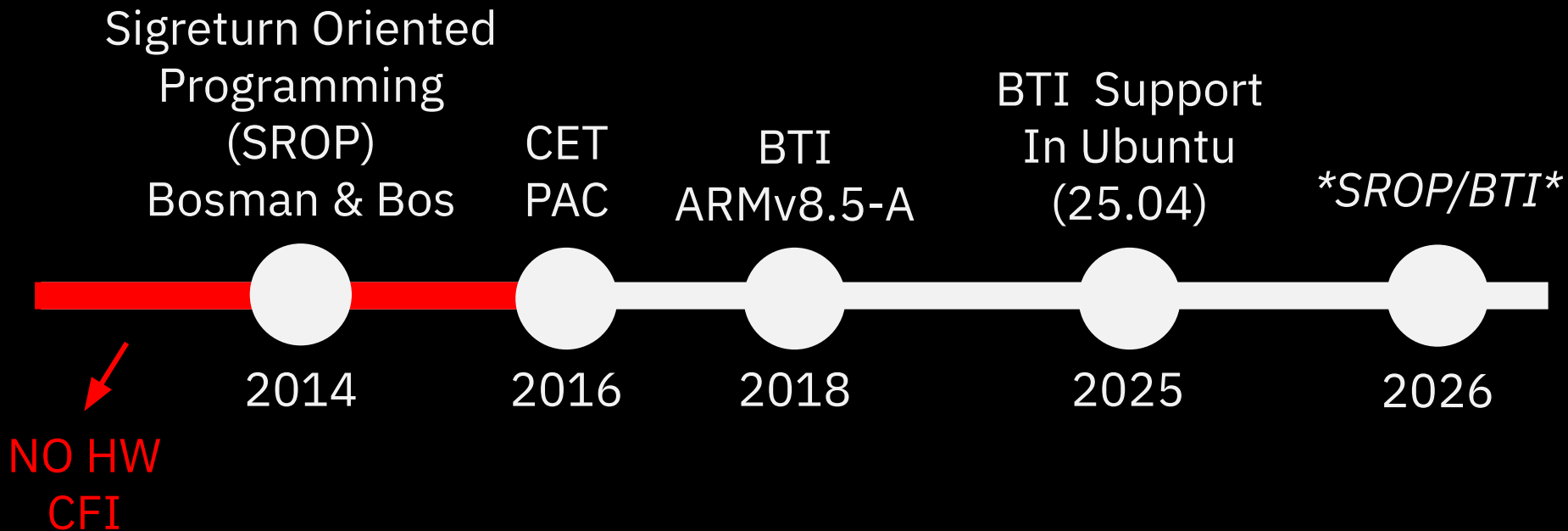
Sigreturn: A BTI Bypass by Design



Sigreturn: A BTI Bypass by Design



From Super Gadget to BTI Bypass



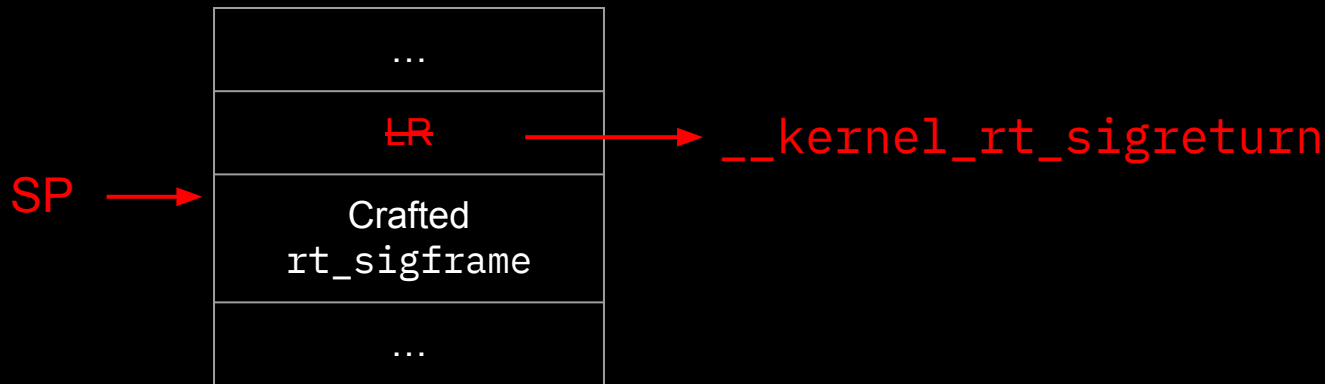
Bootstrapping SROP/BTI

- `rt_sigreturn` takes sigframe from stack pointer
- `pac-ret` $\Rightarrow$ No more (exploitable) stack overflows
- BTI $\Rightarrow$ Can't jump to ROP/JOP stack pivot



Bootstrapping SROP/BTI

- `rt_sigreturn` takes sigframe from stack pointer
- `pac-ret` $\Rightarrow$ No more (exploitable) stack overflows
- BTI $\Rightarrow$ Can't jump to ROP/JOP stack pivot



SROP/BTI CFI-Safe Stack Pivots

1. bti

2. sp := xN // xN=&frame_1

3. br xM // xM=__kernel_rt_sigreturn


```
Listing: libc.so.6 - (16 addresses selected)

0013ec7f d5      ??      D5h

*****
*                               *
*                               *
*****
FUNCTION
*****
undefined setcontext()
undefined <UNASSIGNED> <RETURN>
setcontext
XREF[5]: Entry Point(*),
FUN_0013edc0:0013edcc(j),
swapcontext:0014bad8(c),
00274b28, 0027f570(*)

0013ec80 5f 24 03 d5    bti      c ← Landing Pad
0013ec84 e9 03 00 aa    mov     x9,x0
0013ec88 40 00 80 d2    mov     x0,#0x2
0013ec8c 21 a1 00 91    add     x1,x9,#0x28
0013ec90 02 00 80 d2    mov     x2,#0x0
0013ec94 03 01 80 d2    mov     x3,#0x8
0013ec98 e8 10 80 d2    mov     x8,#0x87
0013ec9c 01 00 00 d4    svc     0x0
0013eca0 40 00 00 b4    cbz     x0,LAB_0013eca8
0013eca4 1f 91 ff 17    b       FUN_00123120
-- Flow Override: CALL_RETURN (CALL_TERMINATOR)
undefined FUN_00123120()

LAB_0013eca8
XREF[1]: 0013eca0(j)
0013eca8 ed 03 1e aa    mov     x13,x30
0013ecac e5 e6 02 94    bl      FUN_001f8840
0013ecb0 fe 03 0d aa    mov     x30,x13
0013ecb4 e0 03 09 aa    mov     x0,x9
0013ecb8 12 cc 54 a9    ldp     x18,x19,[x0, #0x148]
0013ecbc 14 d4 55 a9    ldp     x20,x21,[x0, #0x158]
0013ecc0 16 dc 56 a9    ldp     x22,x23,[x0, #0x168]
0013ecc4 18 e4 57 a9    ldp     x24,x25,[x0, #0x178]
0013ecc8 1a ec 58 a9    ldp     x26,x27,[x0, #0x188]
0013eccc 1c f4 59 a9    ldp     x28,x29,[x0, #0x198]
0013ecd0 1e d4 40 f9    ldr     x30,[x0, #0x1a8] ← LR
0013ecd4 02 d8 40 f9    ldr     x2,[x0, #0x1b0] ← SP
0013ecd8 5f 00 00 91    mov     sp,x2
0013ecdC 02 40 07 91    add     x2,x0,#0x1d0
0013ecdc 23 00 00 f3    mov     x3,#0x0001

LAB_0013ed90
XREF[4]: 0013ecf0(j), 0013ed24(j),
0013ed40(j), 0013ed80(j)
0013ed90 10 dc 40 f9    ldr     x16,[x0, #0x1b8]
0013ed94 02 8c 4c a9    ldp     x2,x3,[x0, #0xc8]
0013ed98 04 94 4d a9    ldp     x4,x5,[x0, #0xd8]
0013ed9c 06 9c 4e a9    ldp     x6,x7,[x0, #0xe8]
0013eda0 00 84 4b a9    ldp     x0,x1,[x0, #0xb8]
0013eda4 00 02 1f d6    br      x16
0013eda8 1c 00 00 17    b       FUN_00123120
```

Glibc CFI Safe Pivot



Caveats

```
63
64  /*
65   * GDB, libgcc and libunwind rely on being able to identify the sigreturn
66   * instruction sequence to unwind from signal handlers. We cannot, therefore,
67   * use SYM_FUNC_START() here, as it will emit a BTI C instruction and break the
68   * unwinder. Thankfully, this function is only ever called from a RET and so
69   * omitting the landing pad is perfectly fine.
70   */
71  SYM_CODE_START(__kernel_rt_sigreturn)
72  //      PLEASE DO NOT MODIFY
73      mov     x8, #__NR_rt_sigreturn
74  //      PLEASE DO NOT MODIFY
75      svc     #0
76  //      PLEASE DO NOT MODIFY
77  //      .cfi_endproc
78  SYM_CODE_END(__kernel_rt_sigreturn)
79
80  emit_aarch64_feature_1_and
```

Caveats

```
63
64  /*
* instruction sequence to unwind from signal handlers. We cannot, therefore,
* use SYM_FUNC_START() here, as it will emit a BTI C instruction and break the
* unwinder. Thankfully, this function is only ever called from a RET and so
* omitting the landing pad is perfectly fine.
*/
71  SYM_CODE_START(__kernel_rt_sigreturn)
72  //      PLEASE DO NOT MODIFY
73      mov     x8, #__NR_rt_sigreturn
74  //      PLEASE DO NOT MODIFY
75      svc     #0
76  //      PLEASE DO NOT MODIFY
77  //      .cfi_endproc
78  SYM_CODE_END(__kernel_rt_sigreturn)
79
80  emit_aarch64_feature_1_and
```


Caveats

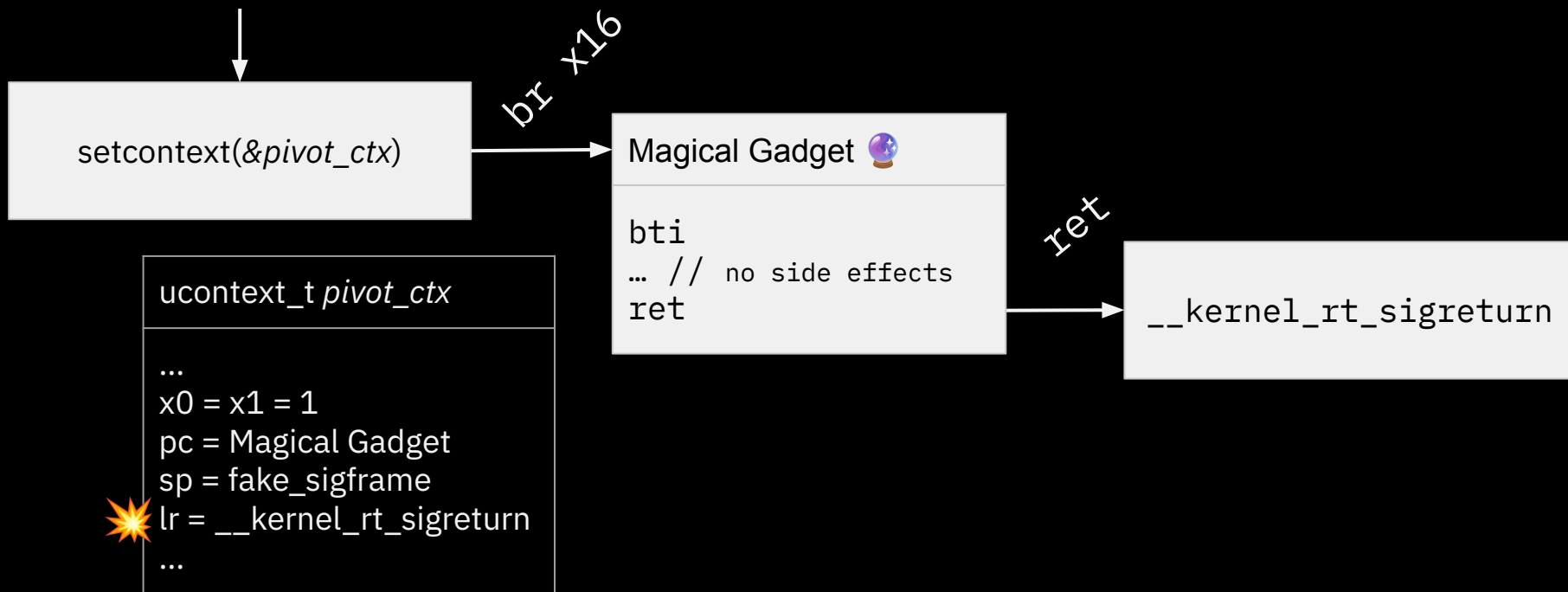
```
63
64  /*
 * instruction sequence to unwind from signal handlers. We cannot, therefore,
 * use SYM_FUNC_START() here, as it will emit a BTI C instruction and break the
 * unwinder. Thankfully, this function is only ever called from a RET and so
 * omitting the landing pad is
```

```
70  */
71  SYM_CODE_START(__kernel_rt_sigreturn)
72  // PLEASE DO NOT REMOVE THIS
73  MOV     X8, #0
74  // PLEASE DO NOT REMOVE THIS
75  SVC     #0
76  // PLEASE DO NOT REMOVE THIS
77  // .cfi_endproc
78  SYM_CODE_END(__kernel_rt_sigreturn)
79
80  emit_aarch64_feature_1_and
```

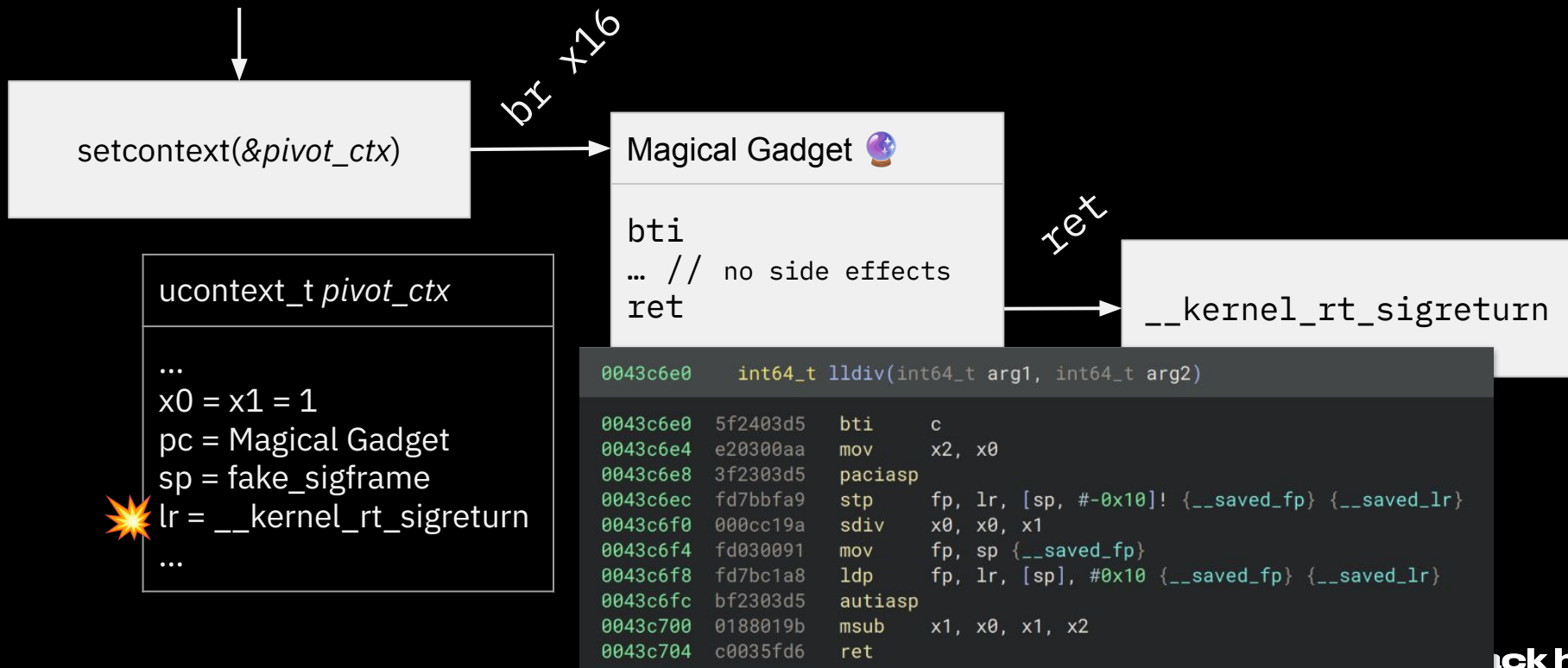
Most
Linux
Distros

```
config ARM64_BTI_KERNEL
bool "Use Branch Target Identification for kernel"
default y
depends on ARM64_BTI
depends on ARM64_PTR_AUTH_KERNEL
depends on CC_HAS_BRANCH_PROT_PAC_RET_BTI
# https://gcc.gnu.org/bugzilla/show_bug.cgi?id=94697
depends on !CC_IS_GCC || GCC_VERSION >= 100100
# https://gcc.gnu.org/bugzilla/show_bug.cgi?id=106671
depends on !CC_IS_GCC
depends on (!FUNCTION_GRAPH_TRACER || DYNAMIC_FTRACE_WITH_ARGS)
help
  Build the kernel with Branch Target Identification annotations
  and enable enforcement of this for kernel code. When this option
  is enabled and the system supports BTI all kernel code including
  modular code must have BTI enabled.
```


Forward-to-Backward Edge Gadget



Forward-to-Backward Edge Gadget



Listing: libc.so - (20 addresses selected)

* FUNCTION *

undefined _libunwind_Registers_arm64_jumpto()
undefined <UNASSIGNED> <RETURN>

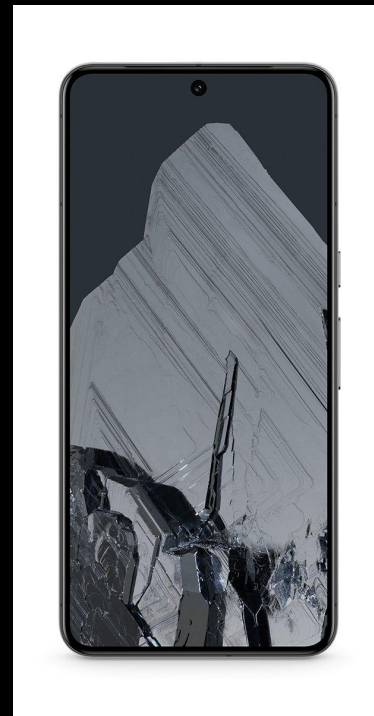
_libunwind_Registers_arm64_jumpto

XREF[3]: _libunwind_shstk_get_jump_targe...
jumpto:001f0910(c), 001ff210(*)

```
001f4190 df 24 03 d5 bti jc
001f4194 02 0c 41 a9 ldp x2,x3,[x0,#0x10]
001f4198 04 14 42 a9 ldp x4,x5,[x0,#0x20]
001f419c 06 1c 43 a9 ldp x6,x7,[x0,#0x30]
001f41a0 08 24 44 a9 ldp x8,x9,[x0,#0x40]
001f41a4 0a 2c 45 a9 ldp x10,x11,[x0,#0x50]
001f41a8 0c 34 46 a9 ldp x12,x13,[x0,#0x60]
001f41ac 0e 3c 47 a9 ldp x14,x15,[x0,#0x70]
001f41b0 12 4c 49 a9 ldp x18,x19,[x0,#0x90]
001f41b4 14 54 4a a9 ldp x20,x21,[x0,#0xa0]
001f41b8 16 5c 4b a9 ldp x22,x23,[x0,#0xb0]
001f41bc 18 64 4c a9 ldp x24,x25,[x0,#0xc0]
001f41c0 1a 6c 4d a9 ldp x26,x27,[x0,#0xd0]
001f41c4 1c 74 4e a9 ldp x28,x29,[x0,#0xe0]
001f41c8 1e 80 40 f9 ldr x30,[x0,#0x100]
001f41cc 00 04 51 6d ldp d0,d1,[x0,#0x110]
001f41d0 02 0c 52 6d ldp d2,d3,[x0,#0x120]
001f41d4 04 14 53 6d ldp d4,d5,[x0,#0x130]
001f41d8 06 1c 54 6d ldp d6,d7,[x0,#0x140]
001f41dc 08 24 55 6d ldp d8,d9,[x0,#0x150]
001f41e0 0a 2c 56 6d ldp d10,d11,[x0,#0x160]
001f41e4 0c 34 57 6d ldp d12,d13,[x0,#0x170]
001f41e8 0e 3c 58 6d ldp d14,d15,[x0,#0x180]
001f41ec 10 44 59 6d ldp d16,d17,[x0,#0x190]
001f41f0 12 4c 5a 6d ldp d18,d19,[x0,#0x1a0]
001f41f4 14 54 5b 6d ldp d20,d21,[x0,#0x1b0]
001f41f8 16 5c 5c 6d ldp d22,d23,[x0,#0x1c0]
001f41fc 18 64 5d 6d ldp d24,d25,[x0,#0x1d0]
001f4200 1a 6c 5e 6d ldp d26,d27,[x0,#0x1e0]
001f4204 1c 74 5f 6d ldp d28,d29,[x0,#0x1f0]
001f4208 1e 00 41 fd ldr d30,[x0,#0x200]
001f420c 1f 04 41 fd ldr d31,[x0,#0x208]
001f4210 10 7c 40 f9 ldr x16,[x0,#0xf8]
001f4214 00 04 40 a9 ldp x0,x1,[x0]
001f4218 1f 02 00 91 mov sp,x16
001f421c 30 00 80 d2 mov x16,#0x1
001f4220 1f 25 03 d5 hint 0x28
001f4224 50 00 00 b5 cbnz x16,Lnogcs
001f4228 1e 77 0b d5 sys 0x3, 0x7, 0x7, 0x0, x30
```

Lnogcs XREF[1]: 001f4224(j)
001f422c c0 03 5f d6 ret

Bionic (/ Ilvm-libunwind) CFI Safe Pivot



Source: Google.

Listing: libc.so - (20 addresses selected)

FUNCTION

undefined _libunwind_Registers_arm64_jumpro()

<UNASSIGNED> <RETURN>

_libunwind_Registers_arm64_jumpro

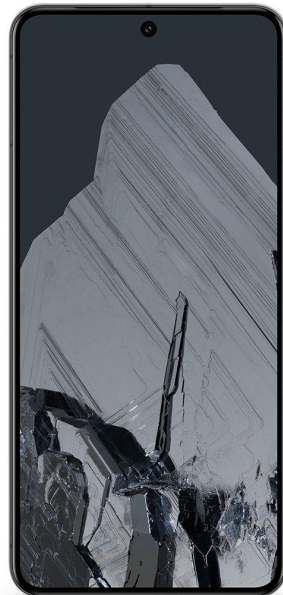
XREF[3]: _libunwind_shstk_get_jump_targe...
jumpro:001f0910(c), 001ff210(*)

```
001f4190 df 24 03 d5 bti jc
001f4194 02 0c 41 a9 ldp x2,x3,[x0,#0x10]
001f4198 04 14 42 a9 ldp x4,x5,[x0,#0x20]
001f419c 06 1c 43 a9 ldp x6,x7,[x0,#0x30]
001f41a0 08 24 44 a9 ldp x8,x9,[x0,#0x40]
001f41a4 0a 2c 45 a9 ldp x10,x11,[x0,#0x50]
001f41a8 0c 34 46 a9 ldp x12,x13,[x0,#0x60]
001f41ac 0e 3c 47 a9 ldp x14,x15,[x0,#0x70]
001f41b0 12 4c 49 a9 ldp x18,x19,[x0,#0x90]
001f41b4 14 54 4a a9 ldp x20,x21,[x0,#0xa0]
001f41b8 16 5c 4b a9 ldp x22,x23,[x0,#0xb0]
001f41bc 18 64 4c a9 ldp x24,x25,[x0,#0xc0]
001f41c0 1a 6c 4d a9 ldp x26,x27,[x0,#0xd0]
001f41c4 1c 74 4e a9 ldp x28,x29,[x0,#0xe0]
001f41c8 1e 80 40 f9 ldr x30,[x0,#0x100]
001f41cc 00 04 51 6d ldp d0,d1,[x0,#0x110]
001f41d0 02 0c 52 6d ldp d2,d3,[x0,#0x120]
001f41d4 04 14 53 6d ldp d4,d5,[x0,#0x130]
001f41d8 06 1c 54 6d ldp d6,d7,[x0,#0x140]
001f41dc 08 24 55 6d ldp d8,d9,[x0,#0x150]
001f41e0 0a 2c 56 6d ldp d10,d11,[x0,#0x160]
001f41e4 0c 34 57 6d ldp d12,d13,[x0,#0x170]
001f41e8 0e 3c 58 6d ldp d14,d15,[x0,#0x180]
001f41ec 10 44 59 6d ldp d16,d17,[x0,#0x190]
001f41f0 12 4c 5a 6d ldp d18,d19,[x0,#0x1a0]
001f41f4 14 54 5b 6d ldp d20,d21,[x0,#0x1b0]
001f41f8 16 5c 5d 6d ldp d22,d23,[x0,#0x1c0]
001f41fc 18 64 5d 6d ldp d24,d25,[x0,#0x1d0]
001f4200 1a 6c 5e 6d ldp d26,d27,[x0,#0x1e0]
001f4204 1c 74 5f 6d ldp d28,d29,[x0,#0x1f0]
001f4208 1e 00 41 fd ldr d30,[x0,#0x200]
001f420c 1f 04 41 fd ldr d31,[x0,#0x208]
001f4210 10 7c 40 f9 ldr x16,[x0,#0xf8]
001f4214 00 04 40 a9 ldp x0,x1,[x0]
001f4218 1f 02 00 91 mov sp,x16
001f421c 30 00 80 d2 mov x16,#0x1
001f4220 1f 25 03 d5 hint 0x28
001f4224 50 00 00 b5 cbnz x16,Lnogcs
001f4228 1e 77 0b d5 sys 0x3, 0x7, 0x7, 0x0, x30
```

Landing Pad

SP

Bionic (/ Ilvm-libunwind) CFI Safe Pivot



Source: Google.

Listing: libc.so - (20 addresses selected)

FUNCTION

undefined _libunwind_Registers_arm64_jumpto()
undefined <UNASSIGNED> <RETURN>

_libunwind_Registers_arm64_jumpto

XREF[3]: _libunwind_shstk_get_jump_targe...
jumpto:001f0910(c), 001ff210(*)

```
001f4190 df 24 03 d5 bti jc
001f4194 02 0c 41 a9 ldp x2,x3,[x0,#0x10]
001f4198 04 14 42 a9 ldp x4,x5,[x0,#0x20]
001f419c 06 1c 43 a9 ldp x6,x7,[x0,#0x30]
001f41a0 08 24 44 a9 ldp x8,x9,[x0,#0x40]
001f41a4 0a 2c 45 a9 ldp x10,x11,[x0,#0x50]
001f41a8 0c 34 46 a9 ldp x12,x13,[x0,#0x60]
001f41ac 0e 3c 47 a9 ldp x14,x15,[x0,#0x70]
001f41b0 12 4c 49 a9 ldp x18,x19,[x0,#0x90]
001f41b4 14 54 4a a9 ldp x20,x21,[x0,#0xa0]
001f41b8 16 5c 4b a9 ldp x22,x23,[x0,#0xb0]
001f41bc 18 64 4c a9 ldp x24,x25,[x0,#0xc0]
001f41c0 1a 6c 4d a9 ldp x26,x27,[x0,#0xd0]
001f41c4 1c 74 4e a9 ldp x28,x29,[x0,#0xe0]
001f41c8 1e 80 40 f9 ldr x30,[x0,#0x100]
001f41cc 00 04 51 6d ldp d0,d1,[x0,#0x110]
001f41d0 02 0c 52 6d ldp d2,d3,[x0,#0x120]
001f41d4 04 14 53 6d ldp d4,d5,[x0,#0x130]
001f41d8 06 1c 54 6d ldp d6,d7,[x0,#0x140]
001f41dc 08 24 55 6d ldp d8,d9,[x0,#0x150]
001f41e0 0a 2c 56 6d ldp d10,d11,[x0,#0x160]
001f41e4 0c 34 57 6d ldp d12,d13,[x0,#0x170]
001f41e8 0e 3c 58 6d ldp d14,d15,[x0,#0x180]
001f41ec 10 44 59 6d ldp d16,d17,[x0,#0x190]
001f41f0 12 4c 5a 6d ldp d18,d19,[x0,#0x1a0]
001f41f4 14 54 5b 6d ldp d20,d21,[x0,#0x1b0]
001f41f8 16 5c 5c 6d ldp d22,d23,[x0,#0x1c0]
001f41fc 18 64 5d 6d ldp d24,d25,[x0,#0x1d0]
001f4200 1a 6c 5e 6d ldp d26,d27,[x0,#0x1e0]
001f4204 1c 74 5f 6d ldp d28,d29,[x0,#0x1f0]
001f4208 1e 00 41 fd ldr d30,[x0,#0x200]
001f420c 1f 04 41 fd ldr d31,[x0,#0x208]
001f4210 10 7c 40 f9 ldr x16,[x0,#0xf8]
001f4214 00 04 40 a9 ldp x0,x1,[x0]
001f4218 1f 02 00 91 mov sp,x16
001f421c 30 00 80 d2 mov x16,#0x1
001f4220 1f 25 03 d5 hint 0x28
001f4224 50 00 00 b5 cbnz x16,Lnogcs
001f4228 1e 77 0b d5 sys 0x3, 0x7, 0x7, 0x0, x30
```

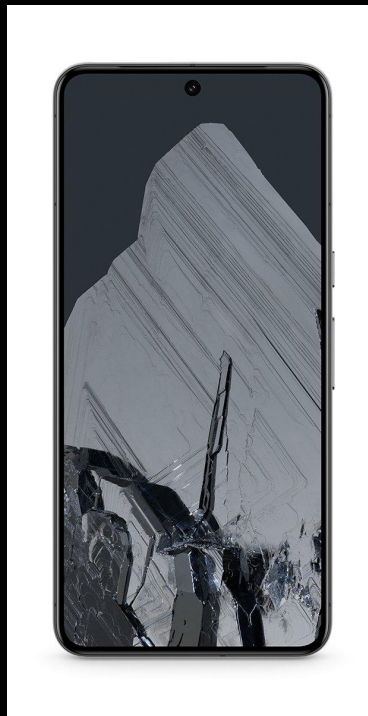
Landing Pad

LR

SP

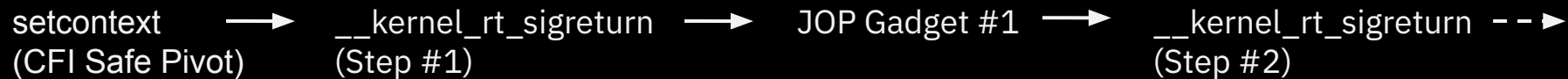
Return to __kernel_rt_sigreturn

Bionic (/ Ilvm-libunwind) CFI Safe Pivot



Source: Google.

Infinite SROP/BTI Chaining



Infinite SROP/BTI Chaining

setcontext (CFI Safe Pivot) → __kernel_rt_sigreturn (Step #1) → JOP Gadget #1 → __kernel_rt_sigreturn (Step #2) - - ►

<i>ucontext_t pivot_ctx</i>
...
pc = __kernel_rt_sigreturn
sp = &frame_1
...



Infinite SROP/BTI Chaining

setcontext (CFI Safe Pivot) → __kernel_rt_sigreturn (Step #1) → JOP Gadget #1 → __kernel_rt_sigreturn --> (Step #2)

<code>ucontext_t pivot_ctx</code>
...
<code>pc = __kernel_rt_sigreturn</code>
<code>sp = &frame_1</code>
...

<code>rt_sigframe frame_1</code>
...
<code>pc = &gadget_1</code>
<code>sp = &frame_2</code>
<code>pstate = 0</code>

<code>Gadget 1 (mem-store)</code>
<code>str w2, [x0] br x16</code>

Infinite SROP/BTI Chaining

setcontext (CFI Safe Pivot) → __kernel_rt_sigreturn (Step #1) → JOP Gadget #1 → __kernel_rt_sigreturn (Step #2) - - ►

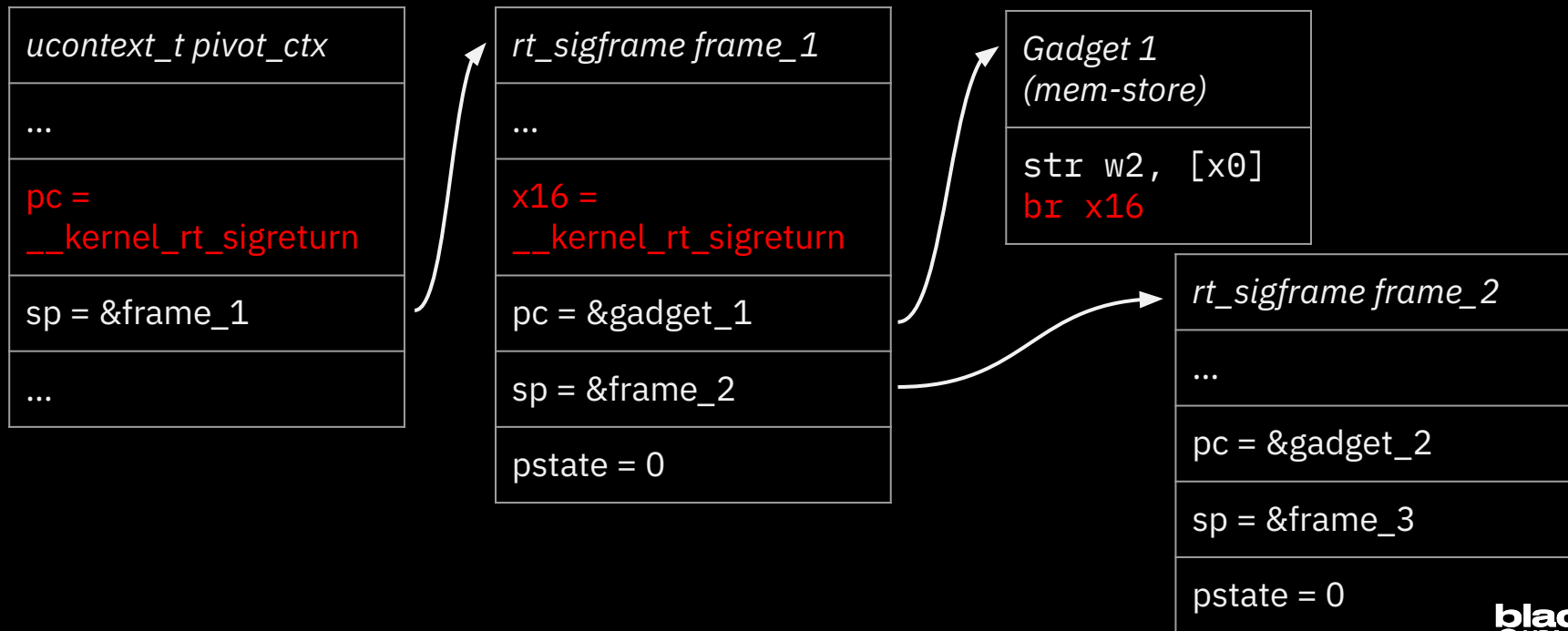
<i>ucontext_t pivot_ctx</i>
...
pc = __kernel_rt_sigreturn
sp = &frame_1
...

<i>rt_sigframe frame_1</i>
...
x16 = __kernel_rt_sigreturn
pc = &gadget_1
sp = &frame_2
pstate = 0

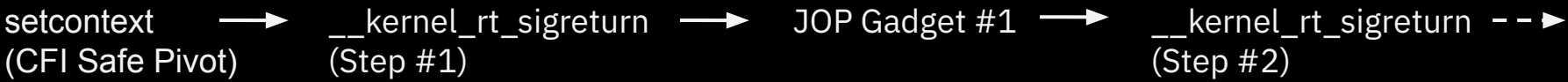
<i>Gadget 1 (mem-store)</i>
str w2, [x0] br x16

Infinite SROP/BTI Chaining

setcontext (CFI Safe Pivot) → __kernel_rt_sigreturn (Step #1) → JOP Gadget #1 → __kernel_rt_sigreturn (Step #2) →



Infinite SROP/BTI Chaining



<i>ucontext_t pivot_ctx</i>
...
pc = __kernel_rt_sigreturn
sp = &frame_1
...

<i>rt_sigframe frame_1</i>
...
x16 = __kernel_rt_sigreturn
pc = &gadget_1
sp = &frame_2
pstate = 0

Gadget 1 (mem-store)
str w2, [x0] br x16

<i>rt_sigframe frame_2</i>
...
pc = &gadget_2
sp = &frame_3
pstate = 0



PoC #1

<https://github.com/betab0t/srop-bti>

Turing Completeness

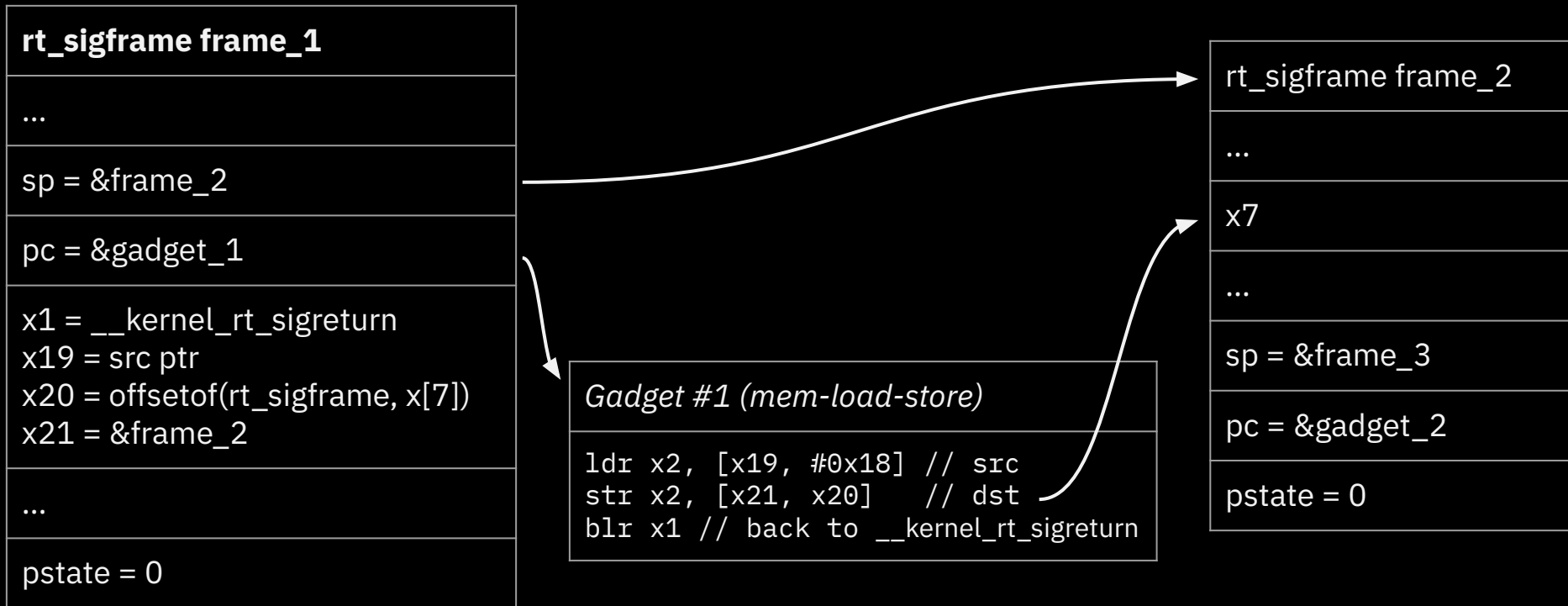
1. Instruction pointer
2. State propagation
3. Operations (load, store, etc')

Instruction Pointer

- Instructions encoded as `rt_sigframes`
- `sp` points to next instruction
- TM steps with `__kernel_rt_sigreturn`

State Propagation

```
((void (*)(int))target)(*(int *)p);
```



State Propagation

```
((void (*)(int))target)(*(int *)p);
```

Gadget #1 (mem-load-store)

```
ldr x2, [x19, #0x18] // src  
str x2, [x21, x20]   // dst  
blr x1 // back to __kernel_rt_sigreturn
```

rt_sigframe frame_2

...

```
x1 = &target  
x7 = *p
```

...

```
sp = &frame_3
```

```
pc = &gadget_2
```

```
pstate = 0
```

Gadget #2 (func-call)

```
mov x0, x7  
blr x1 // target
```


State Propagation

`((void (*)(int))target)(*(int *)p);` 

Gadget #1 (mem-load-store)

```
ldr x2, [x19, #0x18] // src
str x2, [x21, x20]    // dst
blr x1 // back to __kernel_rt_sigreturn
```

rt_sigframe frame_2

...

```
x1 = &target
x7 = *p
```

...

sp = &frame_3

pc = &gadget_2

pstate = 0

Gadget #2 (func-call)

```
mov x0, x7
blr x1 // target
```

Operations

- mem-load/store
- Arithmetics
- Goto
 - Loops
- Conditional branching

Arithmetics

1. Load
2. Calculate
3. Store
4. Sigreturn

Arithmetics

1. Load
2. Calculate
3. Store
4. Sigreturn

SROP/BTI Gadget (madd)

ROP Gadget (madd-gadget-1)

```
madd x0, x10, x8, x9 //  $x0 = (x10 * x8) + x9$   
ret // return to gadget #2
```



JOP Gadget (madd-gadget-2)

```
str w0, [x21, #0x10]  
mov x0, x19  
blr x20 // back to __kernel_rt_sigreturn
```

Pointer authentication

Armv8.3-A introduces the option of pointer authentication, FEAT_PAC. Pointer authentication can mitigate against ROP attacks.

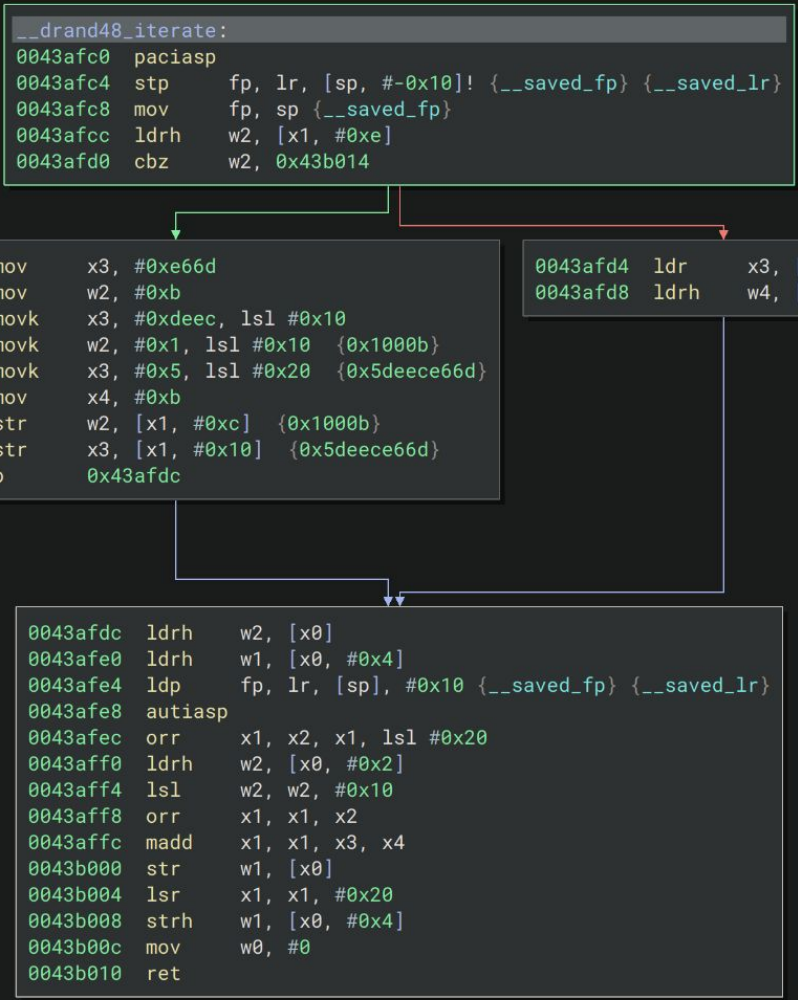
Pointer authentication takes advantage of the fact that pointers are stored in a 64-bit format, but not all those bits are needed to represent the address. This is due to the virtual address space layout:

<https://developer.arm.com/documentation/102433/0200/Return-oriented-programming>

pac

-fschedule-insns2

aut



pac →

```
__drand48_iterate:
0043afc0 paciasp
0043afc4 stp    fp, lr, [sp, #-0x10]! {__saved_fp} {__saved_lr}
0043afc8 mov    fp, sp {__saved_fp}
0043afcc ldrh    w2, [x1, #0xe]
0043afd0 cbz    w2, 0x43b014
```

-fschedule-insns2

```
0043b014 mov    x3, #0xe66d
0043b018 mov    w2, #0xb
0043b01c movk   x3, #0xdeec, lsl #0x10
0043b020 movk   w2, #0x1, lsl #0x10 {0x1000b}
0043b024 movk   x3, #0x5, lsl #0x20 {0x5deece66d}
0043b028 mov    x4, #0xb
0043b02c str    w2, [x1, #0xc] {0x1000b}
0043b030 str    x3, [x1, #0x10] {0x5deece66d}
0043b034 b      0x43afd4
```

```
0043afd4 ldr    x3, [x1, #0x10]
0043afd8 ldrh   w4, [x1, #0xc]
```

aut →

```
0043afdc ldrh    w2, [x0]
0043afe0 ldrh    w1, [x0, #0x4]
0043afe4 ldp     fp, lr, [sp], #0x10 {__saved_fp} {__saved_lr}
0043afe8 autiasp
0043afec orr     x1, x2, x1, lsl #0x20
0043aff0 ldrh    w2, [x0, #0x2]
0043aff4 lsl     w2, w2, #0x10
0043aff8 orr     x1, x1, x2
0043affc madd    x1, x1, x3, x4
0043b000 str     w1, [x0]
0043b004 lsr     x1, x1, #0x20
0043b008 strh    w1, [x0, #0x4]
0043b00c mov     w0, #0
0043b010 ret
```

gadget

pac →

```
__drand48_iterate:
0043afc0 paciasp
0043afc4 stp    fp, lr, [sp, #-0x10]! {__saved_fp} {__saved_lr}
0043afc8 mov    fp, sp {__saved_fp}
0043afcc ldrh    w2, [x1, #0xe]
0043afd0 cbz    w2, 0x43b014
```

-fschedule-insns2
pac-ret[+leaf]

```
0043b014 mov    x3, #0xe66d
0043b018 mov    w2, #0xb
0043b01c movk   x3, #0xdeec, lsl #0x10
0043b020 movk   w2, #0x1, lsl #0x10 {0x1000b}
0043b024 movk   x3, #0x5, lsl #0x20 {0x5deec66d}
0043b028 mov    x4, #0xb
0043b02c str    w2, [x1, #0xc] {0x1000b}
0043b030 str    x3, [x1, #0x10] {0x5deec66d}
0043b034 b      0x43afdc
```

```
0043afd4 ldr    x3, [x1, #0x10]
0043afd8 ldrh   w4, [x1, #0xc]
```

aut →

```
0043afdc ldrh    w2, [x0]
0043afe0 ldrh    w1, [x0, #0x4]
0043afe4 ldp     fp, lr, [sp], #0x10 {__saved_fp} {__saved_lr}
0043afe8 autiasp
0043afec orr     x1, x2, x1, lsl #0x20
0043aff0 ldrh    w2, [x0, #0x2]
0043aff4 lsl     w2, w2, #0x10
0043aff8 orr     x1, x1, x2
0043affc madd    x1, x1, x3, x4
0043b000 str     w1, [x0]
0043b004 lsr     x1, x1, #0x20
0043b008 strh    w1, [x0, #0x4]
0043b00c mov     w0, #0
0043b010 ret
```

gadget

Conditional Branching

- If x Then sp = cond_true Else sp = cond_false

Conditional Branching

- If x Then sp = cond_true Else sp = cond_false
- sp =
$$= \text{cond_true} + (\text{ind}(x) * \text{sizeof}(\text{rt_sigframe}))$$

rt_sigframe frame_branch
sp = ?
pc = __kernel_rt_sigreturn
pstate = 0
rt_sigframe cond_true
rt_sigframe cond_false



Conditional Branching

- If x Then sp = cond_true Else sp = cond_false

- sp =

= cond_true + (ind(x) * sizeof(rt_sigframe))

rt_sigframe frame_branch
sp = ?
pc = __kernel_rt_sigreturn
pstate = 0
rt_sigframe cond_true
rt_sigframe cond_false



```
// ind(x)
ldr w1, [x2]
cmp w1, #0
cset x1, ne
str x1, [sp, #0x10]
// ...
blr x0
```

PoC #2

<https://github.com/betab0t/srop-bti>



Linus Torvalds

To: me, and 2 others, Cc: security@kernel.org, and 3 oth... · Mon, Jan 26 at 8:03 PM ✓

I have forwarded the original email to the arm64 maintainers, also cc'd here.

But:

On Mon, 26 Jan 2026 at 09:37, beta bot <beta_b0t@yahoo.com> wrote:

So this seems to not be a very effective attack vector and is not practically fixable, because signals by *design* have to be able to return anywhere.

This would seem to require that you be able to change the signal return stack in a very particular way, so you probably already had pretty complete control of the program you're attacking.

So this seems to not be a very effective attack vector and is not practically fixable, because signals by *design* have to be able to return anywhere.

I suspect this can be discussed publicly, and the signal return path has - as you point out - already been discussed in various other contexts, but I think this is just how signals work.

BTI isn't some kind of absolute shield. It's just one layer of security protection among many others.

OpenBSD

 openbsd / src ▾

<> **Code** ⓘ Security and quality 📈 Insights

Commit 7730d1d

 **deraadt** committed on May 10, 2016

```
SRP mitigation.  sendsig() stores a (per-process ^ &sigcontext) cookie
inside the sigcontext.  sigreturn(2) checks syscall entry was from the
exact PC addr in the (per-process ASLR) sigtramp, verifies the cookie,
and clears it to prevent sigcontext reuse.
not yet tested on landisk, sparc, *88k, socppc.
ok kettenis
```

 **master**

40 files changed

OpenBSD

 openbsd / src ▾

<> Code ⓘ Security and quality 📈 Insights

Commit 7730d1d

 **deraadt** committed on May 10, 2016 

```
SROP mitigation.  sendsig() stores a (per-process ^ &sigcontext) cookie
inside the sigcontext.  sigreturn(2) checks syscall entry was from the
exact PC addr in the (per-process ASLR) sigtramp, verifies the cookie,
and clears it to prevent sigcontext reuse.
not yet tested on landisk, sparc, *88k, socppc.
ok kettenis
```

 master

40 files changed

XNU

```
1060     if (ts64->pc) {
1061         uint64_t discriminator = ptrauth_string_discriminator("pc");
1062         if (!kernel_signed_pc && userland_diversifier != 0) {
1063             discriminator = ptrauth_blend_discriminator(userland_diversifier,
1064                 ptrauth_string_discriminator("pc"));
1065         }
1066         ts64->pc = (uintptr_t)pmap_auth_user_ptr((void*)ts64->pc,
1067             ptrauth_key_process_independent_code, discriminator,
1068             thread->machine.jop_pid);
1069     }
1070     if (ts64->lr && !(ts64->flags & __DARWIN_ARM_THREAD_STATE64_FLAGS_IB_SIGNED_LR)) {
1071         uint64_t discriminator = ptrauth_string_discriminator("lr");
1072         if (!kernel_signed_lr && userland_diversifier != 0) {
1073             discriminator = ptrauth_blend_discriminator(userland_diversifier,
1074                 ptrauth_string_discriminator("lr"));
1075         }
1076         ts64->lr = (uintptr_t)pmap_auth_user_ptr((void*)ts64->lr,
1077             ptrauth_key_process_independent_code, discriminator,
1078             thread->machine.jop_pid);
1079     }
1080     if (ts64->sp) {
1081         ts64->sp = (uintptr_t)pmap_auth_user_ptr((void*)ts64->sp,
1082             ptrauth_key_process_independent_data, ptrauth_string_discriminator("sp"),
1083             thread->machine.jop_pid);
1084     }
1085     if (ts64->fp) {
1086         ts64->fp = (uintptr_t)pmap_auth_user_ptr((void*)ts64->fp,
1087             ptrauth_key_process_independent_data, ptrauth_string_discriminator("fp"),
1088             thread->machine.jop_pid);
1089     }
1090 }
```

<https://github.com/apple-oss-distributions/xnu/blob/main/osfmk/arm64/status.c#L1060>

 apple-oss-distributions / xnu ▾

<> Code ⓘ Security and quality 🔍 Insights

Commit a5e7219

 AppleOSSDistributions committed on Oct 6, 2021

xnu-6153.11.26

Imported from xnu-6153.11.26.tar.gz

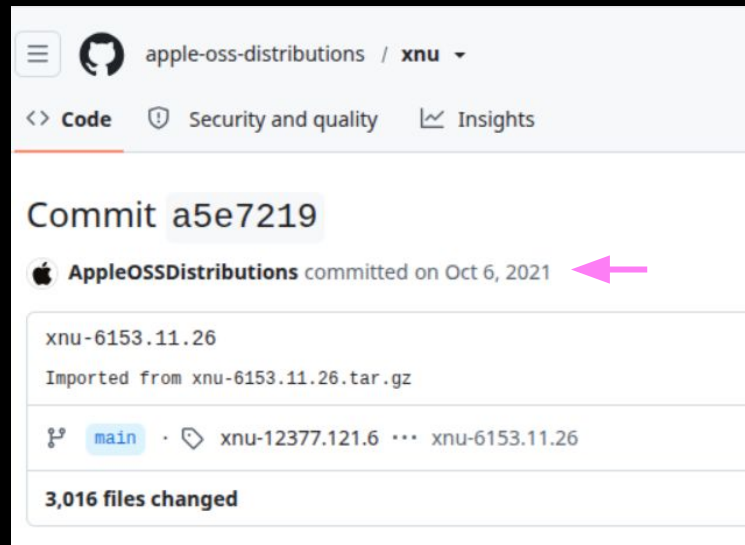
 main ·  xnu-12377.121.6 ... xnu-6153.11.26

3,016 files changed

XNU

```
1060     if (ts64->pc) {
1061         uint64_t discriminator = ptrauth_string_discriminator("pc");
1062         if (!kernel_signed_pc && userland_diversifier != 0) {
1063             discriminator = ptrauth_blend_discriminator(userland_diversifier,
1064                 ptrauth_string_discriminator("pc"));
1065         }
1066         ts64->pc = (uintptr_t)pmap_auth_user_ptr((void*)ts64->pc,
1067             ptrauth_key_process_independent_code, discriminator,
1068             thread->machine.jop_pid);
1069     }
1070     if (ts64->lr && !(ts64->flags & __DARWIN_ARM_THREAD_STATE64_FLAGS_IB_SIGNED_LR)) {
1071         uint64_t discriminator = ptrauth_string_discriminator("lr");
1072         if (!kernel_signed_lr && userland_diversifier != 0) {
1073             discriminator = ptrauth_blend_discriminator(userland_diversifier,
1074                 ptrauth_string_discriminator("lr"));
1075         }
1076         ts64->lr = (uintptr_t)pmap_auth_user_ptr((void*)ts64->lr,
1077             ptrauth_key_process_independent_code, discriminator,
1078             thread->machine.jop_pid);
1079     }
1080     if (ts64->sp) {
1081         ts64->sp = (uintptr_t)pmap_auth_user_ptr((void*)ts64->sp,
1082             ptrauth_key_process_independent_data, ptrauth_string_discriminator("sp"),
1083             thread->machine.jop_pid);
1084     }
1085     if (ts64->fp) {
1086         ts64->fp = (uintptr_t)pmap_auth_user_ptr((void*)ts64->fp,
1087             ptrauth_key_process_independent_data, ptrauth_string_discriminator("fp"),
1088             thread->machine.jop_pid);
1089     }
1090 }
```

<https://github.com/apple-oss-distributions/xnu/blob/main/osfmk/arm64/status.c#L1060>



The screenshot shows the GitHub interface for the repository 'apple-oss-distributions' at the path 'xnu'. The commit being viewed is 'a5e7219', which was committed on October 6, 2021, by AppleOSSDistributions. The commit message is 'xnu-6153.11.26' and it was imported from 'xnu-6153.11.26.tar.gz'. The commit includes a 'main' branch and a range of commits from 'xnu-12377.121.6' to 'xnu-6153.11.26'. A total of 3,016 files were changed in this commit. A pink arrow points to the commit date.

apple-oss-distributions / xnu

<> Code ⓘ Security and quality 🔍 Insights

Commit a5e7219

AppleOSSDistributions committed on Oct 6, 2021

xnu-6153.11.26

Imported from xnu-6153.11.26.tar.gz

main · xnu-12377.121.6 ... xnu-6153.11.26

3,016 files changed

```

1060     if (ts64->pc) {
1061         uint64_t discriminator = ptrauth_string_discriminator("pc");
1062         if (!kernel_signed_pc && userland_diversifier != 0) {
1063             discriminator = ptrauth_blend_discriminator(userland_diversifier,
1064                 ptrauth_string_discriminator("pc"));
1065         }
1066         ts64->pc = (uintptr_t)pmap_auth_user_ptr((void*)ts64->pc,
1067             ptrauth_key_process_independent_code, discriminator,
1068             thread->machine.jop_pid);
1069     }
1070     if (ts64->lr && !(ts64->flags & __DARWIN_ARM_THREAD_STATE64_FLAGS_IB_SIGNED_LR)) {
1071         uint64_t discriminator = ptrauth_string_discriminator("lr");
1072         if (!kernel_signed_lr && userland_diversifier != 0) {
1073             discriminator = ptrauth_blend_discriminator(userland_diversifier,
1074                 ptrauth_string_discriminator("lr"));
1075         }
1076         ts64->lr = (uintptr_t)pmap_auth_user_ptr((void*)ts64->lr,
1077             ptrauth_key_process_independent_code, discriminator,
1078             thread->machine.jop_pid);
1079     }
1080     if (ts64->sp) {
1081         ts64->sp = (uintptr_t)pmap_auth_user_ptr((void*)ts64->sp,
1082             ptrauth_key_process_independent_data, ptrauth_string_discriminator("sp"),
1083             thread->machine.jop_pid);
1084     }
1085     if (ts64->fp) {
1086         ts64->fp = (uintptr_t)pmap_auth_user_ptr((void*)ts64->fp,
1087             ptrauth_key_process_independent_data, ptrauth_string_discriminator("fp"),
1088             thread->machine.jop_pid);
1089     }
1090

```

apple-oss-distributions / xnu

<> Code ⓘ Security and quality 🔍 Insights

Commit a5e7219

AppleOSSDistributions committed on Oct 6, 2021

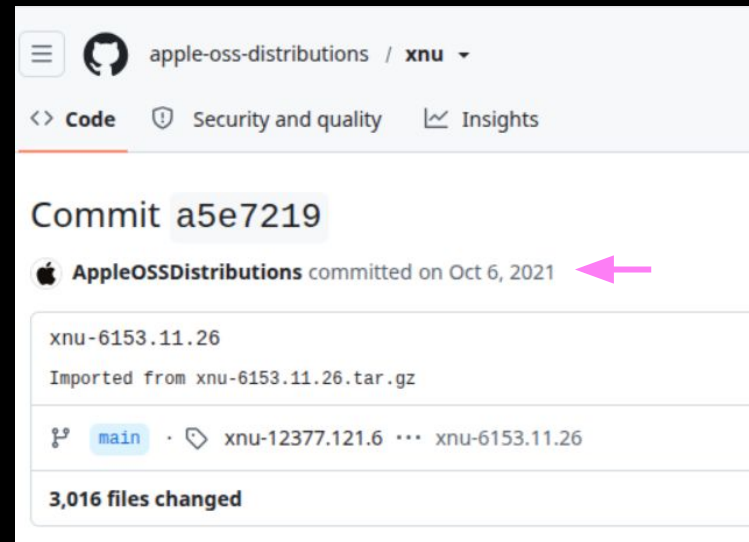
xnu-6153.11.26
Imported from xnu-6153.11.26.tar.gz

main · xnu-12377.121.6 ... xnu-6153.11.26

3,016 files changed

XNU

```
1060     if (ts64->pc) {
1061         uint64_t discriminator = ptrauth_string_discriminator("pc");
1062         if (!kernel_signed_pc && userland_diversifier != 0) {
1063             discriminator = ptrauth_blend_discriminator(userland_diversifier,
1064                 ptrauth_string_discriminator("pc"));
1065         }
1066         ts64->pc = (uintptr_t)pmap_auth_user_ptr((void*)ts64->pc,
1067             ptrauth_key_process_independent_code, discriminator,
1068             thread->machine.jop_pid);
1069     }
1070     if (ts64->lr && !(ts64->flags & __DARWIN_ARM_THREAD_STATE64_FLAGS_IB_SIGNED_LR)) {
1071         uint64_t discriminator = ptrauth_string_discriminator("lr");
1072         if (!kernel_signed_lr && userland_diversifier != 0) {
1073             discriminator = ptrauth_blend_discriminator(userland_diversifier,
1074                 ptrauth_string_discriminator("lr"));
1075         }
1076         ts64->lr = (uintptr_t)pmap_auth_user_ptr((void*)ts64->lr,
1077             ptrauth_key_process_independent_code, discriminator,
1078             thread->machine.jop_pid);
1079     }
1080     if (ts64->sp) {
1081         ts64->sp = (uintptr_t)pmap_auth_user_ptr((void*)ts64->sp,
1082             ptrauth_key_process_independent_data, ptrauth_string_discriminator("sp"),
1083             thread->machine.jop_pid);
1084     }
1085     if (ts64->fp) {
1086         ts64->fp = (uintptr_t)pmap_auth_user_ptr((void*)ts64->fp,
1087             ptrauth_key_process_independent_data, ptrauth_string_discriminator("fp"),
1088             thread->machine.jop_pid);
1089     }
1090 }
```



apple-oss-distributions / xnu

<> Code ⓘ Security and quality 🔍 Insights

Commit a5e7219

AppleOSSDistributions committed on Oct 6, 2021

xnu-6153.11.26
Imported from xnu-6153.11.26.tar.gz

main · xnu-12377.121.6 ... xnu-6153.11.26

3,016 files changed



PRACTICALLY FIXABLE

<https://github.com/apple-oss-distributions/xnu/blob/main/osfmk/arm64/status.c#L1060>

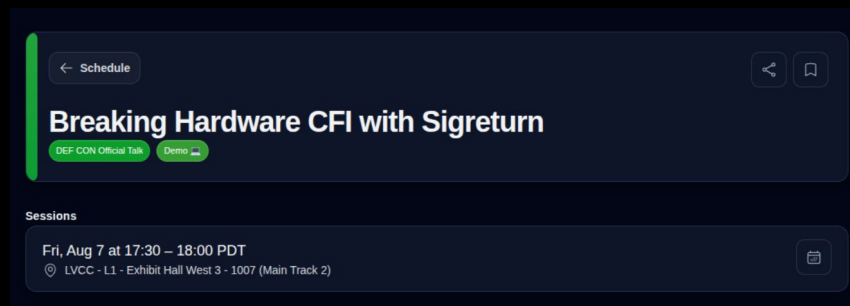
Omri Ben-Bassat “beta_b0t”

beta_b0t@yahoo.com

<https://www.linkedin.com/in/omri-ben-bassat>



Thank You



<https://hackertracker.app/defcon34/content/66610>

